

AWS Lambda, carte mentale

PIÈGE classique à l'examen **PRÉCISION** corrigé ou complète tes notes **Sigles soulignés en pointillé** : voir le glossaire en bas

Lambda
Du code sans serveur, déclenché par des événements

01 Serverless et Lambda

POURQUOI LE SERVERLESS

- Le cloud permet d'abstraire la couche infrastructure pour se concentrer sur la **logique métier**
- Niveaux d'abstraction : on-premises → serveurs virtuels → conteneurs → **serverless**
- Serveurs classiques : tu gères les instances, les correctifs de l'OS, la plateforme, le scaling, la sécurité. Serverless : tu construis, déploies et surveilles
- Tu paies le temps d'exécution du code, pas la capacité au repos
- Principe moderne : **petites pièces faiblement couplées**, bases de données dédiées, services managés pour l'intégration, infrastructure as code
- Monolithe : simple au départ, mais une seule pile techno, une seule base dimensionnée pour le pire cas, un pipeline de livraison partagé. Microservices : chaque pièce évolue seule, au prix d'une courbe d'apprentissage

LAMBDA

- Service de calcul **serverless** : exécute ton code sans provisionner ni gérer de serveur
- Invoke** ton code en réponse à des événements
- Scale** automatiquement
- Logs et métriques intégrés via CloudWatch
- Païement au temps de calcul utilisé, rien au repos
- Ton code : Node.js, Python, Ruby, Java, Go, .NET, ou un **runtime personnalisé**

DEUX COMPOSANTS

- Event source** : ce qui publie des événements vers Lambda
- Function** : ton code, exécuté quand la source déclenche

LES 6 ATOUTS

- Apporter son propre code
- S'intègre aux autres services AWS et les étend
- Modèles flexibles de ressources et de concurrence
- Modèle de permissions flexible
- Disponibilité et tolérance aux pannes intégrées
- Plus de ressources inactives à payer

CAS D'USAGE

- Application web** : S3 (front statique) → API Gateway → Lambda → DynamoDB → réponse
- Flux temps réel** : Kinesis → Lambda traite les nouveaux records → résultats dans DynamoDB
- Backends** web, mobile, IoT, API tierces
- Traitement de données** à chaque changement d'état : upload S3, écriture DynamoDB, alarme CloudWatch, horaire
- Chatbots** (Lex) et skills **Alexa**
- Automatisation IT** : démarrer/arrêter des EC2, mettre à jour des security groups

02 La fonction et la concurrence

CE QUI COMPOSE UNE FONCTION

- Permissions d'accès** : qui l'invoque, ce qu'elle a le droit de faire
- Événements déclencheurs**
- Runtime et package de déploiement** : ton code et ses dépendances
- Configuration** : mémoire, timeout, variables d'environnement...

ENVIRONNEMENTS ÉPHÉMÈRES

- Chaque invocation tourne dans un environnement créé à la demande, de courte durée, puis supprimé
- Tu ne le contrôles pas : Lambda gère son cycle de vie

CONCURRENCE

- Concurrence** = nombre d'invocations qui tournent **en même temps**. Lambda crée de nouveaux environnements pour suivre le rythme
- Reserved concurrency** : plafond optionnel **par fonction**, pris sur le quota régional. Empêche une fonction d'affamer les autres ou de submerger un système en aval
- Quota régional** : limite souple du total des invocations simultanées, **1 000** par défaut par compte et par Région
- Montée en charge** : limite sur la vitesse à laquelle la concurrence peut grimper lors d'un pic
- Cadence des requêtes > durée de chaque exécution
- La **source d'événements** : chaque service gère à sa façon la concurrence de ses invocations

- PRÉCISION** Depuis fin 2023, chaque fonction monte de **1 000** environnements toutes les **10 s** (jusqu'au quota du compte). Les anciens « burst limits » régionaux ne s'appliquent plus tels quels

03 Invocation

push synchrone, push asynchrone ou pull

LES 3 MODES

	PUSH SYNCHRONE	PUSH ASYNCHRONE	PULL (POLLING)
Qui appelle	La source invoque Lambda directement, l'appelant attend la réponse	La source dépose l'événement, Lambda le met en file, la source repart (« envoi et oublie »)	La source met l'info dans un stream ou une file ; Lambda va la chercher et invoque en lots
Invocation type	RequestResponse	Event	Via un event source mapping
Statut HTTP	200	202 (accepté, pas « réussi »)	n/a
Retry	Aucun côté Lambda : c'est ton code qui réessaie	0 à 2 retries automatiques, puis DLQ ou destination OnFailure	Selon la source (voir plus bas)
Exemples	API Gateway, ELB (ALB), Cognito, Lex, Alexa, CloudFront, Firehose	S3, SNS, SES, CloudWatch Logs, CloudWatch Events / EventBridge, Config	DynamoDB Streams, Kinesis, SQS

- PIÈGE** Quand un service AWS déclenche Lambda, tu n'as **aucun contrôle** sur le type d'invocation : il est prédéfini par la source

PULL : STREAM OU FILE

- Stream** (DynamoDB Streams, Kinesis) : records organisés en **shards**, Lambda lit un shard et invoque par lots
 - En cas d'échec, Lambda bloque la lecture du shard jusqu'à ce que le lot réussisse ou expire (24 h)
 - Parade : une destination **OnFailure** pour contourner un record qui échoue toujours

- File** (SQS) : en cas d'échec, le message redevient visible et est retenté, jusqu'à la limite de retires ou la rétention. Les messages qui échouent toujours vont dans une **DLQ**

- Le lien source ↔ fonction est un **event source mapping** (CreateEventSourceMapping)

- Le polling lui-même est **gratuit** : tu ne paies que les actions qui en résultent

INVOKER À LA MAIN

```
# Synchronise
aws lambda invoke --function-name my-function \
  --payload '{"...}' response.json
# → {"StatusCode": 200}

# Asynchrone
aws lambda invoke --function-name my-function \
  --invocation-type Event --payload '{"...}' response.json
# → {"StatusCode": 202}
```

- PRÉCISION** Avec l'AWS CLI v2, un payload JSON brut demande --cli-binary-format raw-in-base64-out

04 Permissions

deux questions distinctes, toutes deux via IAM

LE SCHÉMA

Source d'événements → peut invoquer ? **Lambda** → peut agir sur ?

Service ou ressource AWS

QUESTION	RÉPONSE	OÙ
Qui peut m' invoquer ?	Resource policy (lambda:InvokeFunction)	Attachée à la fonction
Que peut faire ma fonction ?	Rôle d'exécution	Rôle IAM, utilisé par la fonction

1. RESOURCE POLICY : QUI PEUT INVOKER

- Donne à une source d'événements l'action **lambda:InvokeFunction**
- Pour une source hors Lambda comme S3, la permission équivalente peut vivre sur l'**autre** ressource (le bucket) plutôt que sur la fonction
- Permet l'invocation **cross-account** (le bucket du compte A invoque une fonction du compte B)

```
{ "Effect": "Allow",
  "Principal": { "Service": "s3.amazonaws.com" },
  "Action": "lambda:InvokeFunction",
  "Resource": "arn:aws:lambda:...:function:ma-fonction",
  "Condition": { "ArnLike": { "AWS:SourceArn": "arn:aws:s3:::mon-bucket" } } }
```

2. RÔLE D'EXÉCUTION : CE QUE LA FONCTION PEUT FAIRE

- Policy IAM** : les actions permises sur d'autres ressources (écrite dans DynamoDB, lire SQS, écrire les logs CloudWatch). « Qui a le droit de faire quoi »

- Trust policy** : quelle entité peut **assumer** le rôle. Sans elle, même si le rôle existe, personne ne peut l'utiliser. « Qui peut l'endosser »

- PRÉCISION** iam:PassRole est exigé de l'**identifié qui crée ou modifie** la fonction (celle qui « passe » le rôle), pas de Lambda elle-même

- PIÈGE** Une source en **polling** (stream, file) utilise le **rôle d'exécution** pour lire, pas une resource policy

```
// Trust policy du rôle
{ "Effect": "Allow",
  "Principal": { "Service": "lambda.amazonaws.com" },
  "Action": "sts:AssumeRole" }
```

05 Le handler

- Point d'entrée que Lambda appelle : **handler(event, context)**
- Chaque langage a sa convention pour le désigner dans le package. Python : **module.fonction** (ex. **app.lambda_handler**)
- event** : ce qui a déclenché la fonction. Sa forme dépend de la source
 - API Gateway : chemin, query string, corps de la requête
 - S3 : bucket et objet concernés
 - Peut aussi être un objet de test que tu définis

- context** : informations d'exécution générées par AWS, variables selon le langage. Au minimum :
 - awsRequestId** (Python : **aws_request_id**) : identifie une invocation, utile pour le support AWS
 - logStreamName** : le log stream CloudWatch de l'invocation
 - getRemainingTimeInMillis()** : ms restantes avant le timeout

06 Configuration

PERFORMANCE

RÉGLAGE	CE QU'IL CONTRÔLE
Mémoire	128 MB à 10 240 MB . Le CPU est proportionnel à la mémoire
Timeout	1 s à 15 min . Évite le coût d'une fonction bloquée
Concurrence	1 000 invocations simultanées par compte et par Région (limite souple). Possibilité de réserver une part à une fonction
Provisioned concurrency	Garde des environnements pré-chauffés pour éviter les cold starts . Facturé à part
Monitoring	X-Ray (active tracing) et CloudWatch Lambda Insights

PRIX

- Facturation : **nombre de requêtes × durée**
- Le prix par milliseconde monte avec la mémoire...
- ...mais plus de mémoire = plus de CPU = durée plus courte. Pour du travail lié au CPU, une mémoire plus haute peut donc coûter **moins cher**
- Utilitaire **power tuning** pour trouver la mémoire qui minimise le coût total

RESSOURCES

RÉGLAGE	CE QU'IL CONTRÔLE
Triggers	Les sources d'événements qui invoquent la fonction
Permissions	Qui l'invoque, ce qu'elle peut atteindre
Destinations	Où va le résultat d'une invocation async, en succès ou en échec : topic SNS, file SQS, autre Lambda, bus EventBridge
Invocation async	Nombre de retries (0 à 2), âge max d'un événement (jusqu'à 6 h), DLQ
VPC	Accès aux ressources d'un VPC privé (bases, caches, services internes)
State machines	Machines Step Functions qui peuvent l'invoquer
Database proxies	RDS Proxy : mise en commun des connexions
File systems	Montage d'un système de fichiers EFS

CODE

- Runtime** : langage et version, ou runtime personnalisé
- Variables d'environnement** : paires clé-valeur lues par le code sans le modifier. Bonnes pour la config et pour des valeurs sensibles non secrètes ou chiffrées avec KMS
- Tags** : suivi et filtrage des coûts
- Code signing** : garantit que le code vient d'une source approuvée et n'a pas été altéré

07 Bonnes pratiques

CONCEVOIR LA FONCTION

- Sans état** : ne jamais supposer qu'une donnée en mémoire survit entre deux invocations

- Seulement le nécessaire** : package et dépendances minimaux, pas tout un SDK pour deux modules. Moins de dépendances = démarrage plus rapide

- Réutiliser l'environnement** : initialiser les ressources coûteuses (connexion à une base, client SDK) **hors du handler**, pour que les invocations « chaudes » les réutilisent

ÉCRIRE LE CODE

- Séparer la logique métier du handler : plus portable, plus facile à tester

- Fonctions modulaires, à usage unique

- Ajouter des logs (écrits dans CloudWatch) et renvoyer un résultat

- Variables d'environnement plutôt que valeurs en dur (nom de bucket ou de table)

- PIÈGE** Pas de code récursif : une fonction qui s'invoque elle-même peut s'emballer via la concurrence

- PIÈGE** Ne pas appeler une Lambda depuis une autre : utiliser des destinations ou orchestrer avec **Step Functions**

08 Déploiement : package, versions, alias, layers

DEUX TYPES DE PACKAGE

	.ZIP	IMAGE CONTENEUR
Principe	Choisir un runtime, compresser code et dépendances, uploader	Choisir runtime et distribution Linux, emballer le code dans une image, la pousser dans ECR
Mise en ligne	Éditeur de la console, package depuis l'IDE, ou zip déposé sur S3 puis référencé	ECR, la tire l'image d'Lambda, l'optimise, la déploie : statut PENDING puis ACTIVE
Limites	50 MB zippé (upload direct) ; 250 MB décompressé avec les layers ; 3 MB dans l'éditeur console ; 512 KB par fichier individuel	Jusqu'à 10 GB (voir branche 11)

CRÉER ET METTRE À JOUR (CLI)

```
aws lambda create-function \
  --function-name my-function --runtime python3.13 \
  --zip-file fileb://my-function.zip \
  --handler app.lambda_handler \
  --role arn:aws:iam::123456789012:role/MonRole

aws lambda update-function-code \
  --function-name my-function \
  --zip-file fileb://my-function.zip
```

- PRÉCISION** L'exemple du cours utilise **nodejs10.x**, un runtime retiré depuis longtemps. Prends un runtime actuellement supporté

VERSIONS ET ALIAS

- Version** : instantané **immuable** du code + de la configuration. Seule **\$LATEST** existe tant qu'on ne publie pas. Chaque version publiée (**1**, **2**, ...) a son propre ARN
- PIÈGE** Une fois publiée, une version ne peut plus être modifiée
- Alias** : pointeur **mutable** et nommé vers une version (DEV, TEST, PROD), avec son propre ARN
- Promouvoir ou revenir en arrière = repointer l'alias, sans changer le code

- PIÈGE** Brancher les services (notification S3, intégration API Gateway) sur l'**alias** et non sur une version : sinon il faut modifier la config de chaque service à chaque déploiement

LAYERS

- Archive .zip de bibliothèques, de runtime personnalisé ou de dépendances partagées, attachable sans les intégrer au package

- Maximum **5** layers par fonction, **250 MB** au total décompressé (fonction + layers)

- Garde les packages petits (éditable dans la console sous 3 MB pour Node.js, Python, Ruby) et partage une dépendance entre plusieurs fonctions

- AWS publie un layer public NumPy / SciPy pour Python

RUNTIMES PERSONNALISÉS

- Pour un langage non supporté. Une **runtime interface** reçoit les événements et envoie les réponses

- Le fichier **bootstrap** est le point d'entrée. Livrable avec le code ou comme layer

09 Surveillance et débogage

CLOUDWATCH

- Logs, métriques et événements dans une vue unifiée. Logs lus comme un flux chronologique

- Logs Insights** : requêtes interactives sur les logs

- Métriques par défaut, **alarmes** sur seuils avec actions automatiques

STRUCTURE DES LOGS LAMBDA

- Un log group par fonction : /aws/lambda/<nom-fonction>, créé à la première invocation (ou à la config de la console concurrency)

- Log streams organisés par date, version de fonction et ID unique

- Chaque invocation a un **RequestId** et 3 lignes clés :
 - START** : début
 - END** : fin
 - REPORT** : durée facturée, mémoire configurée, mémoire utilisée et, si présent, **Init Duration** = cette invocation a payé un cold start

X-RAY

- Suit une requête à travers les composants de l'application et dessine une **service map**

- Activation en un clic pour Lambda ; le SDK X-Ray capte les appels faits avec le SDK AWS

- Distribution visuelle des latences, isolement des valeurs aberrantes, filtrage par type d'erreur

ÉLÉMENTS

Segment	RÔLE
Segment	Travail d'une ressource de calcul : nom, détails de la requête, sous-segments
Subsegment	Découpage plus fin : appel à une base, une API externe, un service AWS, ou un bloc de code
Annotations	Paires clé-valeur indexées , utilisables dans les expressions de filtre pour chercher et grouper
Metadata	Paires de tout type, non indexées , pour du contexte qu'on ne cherchera pas

- PIÈGE** Annotation = on peut chercher dessus. Metadata = non

10 Cas appliqué : le handler du fraud-detector

LES 7 ÉTAPES

- 1. Lire l'event** : avec l'intégration proxy, le corps HTTP arrive comme une chaîne **JSON** dans **event["body"]**, jamais déjà parsé

- 2. Extraire avec .get()** : **body["user_id"]** lève une **KeyError** si le champ manque. **body.get("user_id", "unknown")** dégrade proprement. Le défaut « unknown » est un raccourci temporaire, avant Cognito

- 3. Ne jamais faire confiance au client** pour l'identité ni l'heure : **transaction_id** et **timestamp** sont générés **côté serveur**, sinon un client pourrait fabriquer des IDs, provoquer des collisions ou antider

- 4. Construire les clés** avec de simples f-strings

- 5. Valider avant d'écrire** : échouer vite avec un 4xx clair

- 6. Écrire** : **table.put_item(item=item)**, la seule ligne qui touche DynamoDB. Tout ce qui précède se teste sans base

- 7. Toujours renvoyer une réponse valide** : **statusCode** et **body** au minimum, avec **body** en chaîne **JSON**, pas un dict

LE CODE

```
body = json.loads(event["body"])
user_id = body.get("user_id", "unknown")
amount = body.get("amount")
merchant = body.get("merchant")

if amount is None or merchant is None:
    return build_response(400,
        {"error": "amount and merchant are required"})

transaction_id = str(uuid.uuid4())
timestamp = (datetime.now(tz=timezone.utc)
    .isoformat(timespec="seconds")
    .replace("+00:00", "Z"))

pk = f"USER#{user_id}"
sk = f"TXN#{timestamp}#{transaction_id}"
table.put_item(Item=item)

def build_response(status_code, data):
    return {
        "statusCode": status_code,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "GET,POST,OPTIONS"
        },
        "body": json.dumps(data)
    }
```

- Les en-têtes CORS sont renvoyés par la Lambda elle-même (voir les cartes DynamoDB et API Gateway)

- PRÉCISION** À l'écriture, boto3 refuse les **float** : **TypeError: float types are not supported. Use Decimal types instead**. Parser avec **json.loads(event["body"], parse_float=Decimal)**

11 Hors de tes notes

à connaître pour le DVA-C02

LIMITES À CONNAÎTRE

- Payload d'une invocation **synchrone** : **6 MB** (requête et réponse)

- Payload d'une invocation **asynchrone** : **256 KB** jusqu'en 2025, **1 MB** depuis octobre 2025. Un examen ancien peut encore citer 256 KB

- Stockage éphémère **/tmp** : **512 MB** par défaut, jusqu'à **10 240 MB**. Peut persister entre invocations « chaudes », sans garantie

- Variables d'environnement : **4 KB** au total

- Image conteneur : jusqu'à **10 GB**

- Vitesse de calcul : environ **1 769 MB** de mémoire = 1 vCPU

- Gratuit chaque mois : 1 million de requêtes + 400 000 GB-secondes

ERREURS ET RETRIES, EN DÉTAIL

- PIÈGE** Pour une fonction déclenchée par **SQS**, la DLQ se configure sur la **file source**, pas sur la fonction. Le réglage DLQ de la fonction ne vaut que pour l'invocation **asynchrone**

- Les **destinations** (success / échec) sont préférées à la DLQ : elles portent plus d'informations

- Streams, réglages de l'event source mapping : **BisectBatchOnFunctionError**, **MaximumRetryAttempts**, **MaximumRecordAgeInSeconds**, **ReportBatchItemFailures**

- SQS** : le **visibility timeout** de la file doit valoir au moins **6x** le temps de l'exécution. **Partial batch response** au moins ne retenir que les messages en échec

- Throttling (concurrence atteinte) : erreur **429 TooManyRequestsException** en synchrone ; en asynchrone, Lambda retente pendant jusqu'à 6 h

CONCURRENCE ET DÉMARRAGE

- Reserved concurrency** : sans frais, garantit une capacité **et** plafonne la fonction. La mettre à **0** coupe la fonction (utile en urgence)

- Provisioned concurrency** : payante, entraîne les cold starts

- SnapStart** : réduit le cold start en restaurant un instantané de l'environnement initialisé (Java, et aussi Python et .NET)

- Le cold start dépend de la taille du package, du runtime et de l'initialisation hors handler

RÉSEAU ET SÉCURITÉ

- PIÈGE** Lambda dans un **VPC** : pas d'accès à Internet (ni aux API AWS publiques) sans **NAT gateway** ou **endpoints VPC**. La fonction a aussi besoin des droits pour créer ses interfaces réseau

- PIÈGE** Pas de logs dans CloudWatch ? Le rôle d'exécution n'a pas les droits d'écriture (politique **AWSLambdaBasicExecutionRole**)

- Secrets : variables d'environnement chiffrées avec KMS, ou mieux **Secrets Manager / Parameter Store**

- Appel par API** : la ressource de la fonction doit autoriser **apigateway.amazonaws.com**. Si la fonction est désignée par une variable de stage, ajouter la permission à la main (**aws lambda add-permission**)

DÉPLOIEMENT PROGRESSIF

- Un **alias pondéré** répartit le trafic entre deux versions (ex. 90 % / 10 %) : un canary natif

- SAM** : AutoPublishAlias et DeploymentPreference pilotent CodeDeploy pour des déploiements progressifs

AUTRES DÉCLENCHEURS

- Function URL** : endpoint HTTPS dédié à la fonction, avec auth **AWS_IAM** ou **NONE**, sans API Gateway

- EventBridge** : appel planifié (cron / rate)

MÉTRIQUES CLOUDWATCH UTILES

- **SAM** : AutoPublishAlias et DeploymentPreference pilotent CodeDeploy pour des déploiements progressifs

AUTRES DÉCLENCHEURS

- **Function URL** : endpoint HTTPS dédié à la fonction, avec auth `AWS_IAM` ou `NONE`, sans API Gateway
- **EventBridge** : appel planifié (cron / rate)

MÉTRIQUES CLOUDWATCH UTILES

MÉTRIQUE	SENS
<code>Invocations</code>	Montre le nombre de fois que la fonction est appelée